

In-Network IoT Attack Detection Using Principal Component Analysis and Machine Learning Models on P4 Programmable Data Planes

Viet Anh Phan¹[0009-0003-1787-8063]✉, Mingyu Ma^{2,4}[0009-0005-7045-4096],
Tung V. Doan^{2,4}, Giang T. Nguyen^{2,4}[0000-0001-7008-1537],
Frank H.P. Fitzek^{3,4}[0000-0001-8469-9573], Jan Jerabek¹[0000-0001-9487-5024],
Lukas Malina¹[0000-0002-7208-2514], and Jiri Hosek¹[0000-0002-8382-9185]

¹ Department of Telecommunications, Brno University of Technology, Czech Republic
{243760, jerabekj, malina, hosek}@vut.cz

² Haptic Communication Systems, TU Dresden, Germany

³ Deutsche Telekom Chair of Communication Networks, TU Dresden, Germany

⁴ Centre for Tactile Internet with Human-in-the-Loop (CeTI), TU Dresden, Germany
{mingyu.ma, tung.doan_van, giang.nguyen, frank.fitzek}@tu-dresden.de

Abstract. As network speeds scale toward the multi-terabit range, traditional Intrusion Detection Systems struggle with the latency and bandwidth overhead of offloading traffic to external controllers. Implementing security logic directly within the programmable data plane enables line-rate inference and eliminates the control plane bottleneck. However, the constrained memory and computational resources in the data plane make complex Machine Learning (ML) deployment challenging. To our knowledge, this paper presents the first ML-based intrusion-detection pipeline that integrates dimensionality reduction via Principal Component Analysis (PCA) with a tree-based classifier entirely inside a P4 programmable data plane, encoding both stages as range-match tables. This range-match approach represents each tree leaf as a single rule, in contrast to the exact-match encodings used by prior systems, and reduces the source-level classifier-table description by up to 51.71%. Our system extracts 20 bidirectional flow features at line-rate to classify four IoT traffic categories (Benign, DoS, Brute Force, and Reconnaissance). Evaluations on the CIC-IoT 2023 dataset using the BMv2 switch demonstrate high detection accuracy, with macro F1 scores between 95.45% and 97.07% across 24 PCA configurations. Furthermore, we show that applying PCA reduces the per-entry match-key width by 59% (224 vs 552 bits at $k=7, b=32$) compared with feeding raw features directly to a range-match Decision Tree, and improves robustness to light input perturbations. Finally, a resource analysis on the Intel Tofino 1 architecture shows that our packed PCA layout clears the per-stage match-key and SRAM gates, with end-to-end compilation as ongoing work.

Keywords: P4 programmable data plane · In-network machine learning · IoT · Intrusion detection system · Principal Component Analysis · Decision tree · Random forest · Range match tables · Bidirectional flow features · Line-rate classification

1 Introduction

Network attacks continue to grow in intensity and sophistication, placing stringent demands on detection infrastructure [6]. Traditional Intrusion Detection Systems (IDS) deployed as middleboxes or within Software Defined Networking (SDN) controllers process copies of traffic off the forwarding path, introducing latency from packet mirroring and control-channel overhead, while OpenFlow switches must delegate deep packet inspection, stateful flow tracking, and custom parsing to the controller [6]. The controller therefore becomes a single point of failure that bounds the speed of security decisions [15], a particular problem for IoT environments in which resource-constrained devices generate heterogeneous traffic that must be classified at wire speed. P4 programmable data planes address these limitations by enabling user-defined parsing, stateful register manipulation, and match-action classification inside the switch pipeline [6], motivating a growing body of work that embeds ML-based detection in the data plane by encoding trained classifiers such as Decision Tree (DT) and Random Forest (RF) as match-action entries [16,9].

Despite this progress, the existing body of work reveals five persistent gaps. *First*, most in-network ML systems rely on exact-match encoding, where the classifier table stores one entry per unique discretized training feature vector [9,19,18]. Coverage is therefore restricted to feature combinations actually observed during training: any input that does not appear in the lookup misses, even if it falls inside a region that the underlying tree would classify confidently. The number of entries also scales with the granularity of the discretization grid and the size of the training set N , so adding training samples or refining feature granularity grows the table proportionally and stresses on-chip memory budgets [2]. *Second*, the majority of P4-based detection systems perform only binary classification, which limits their utility for security teams that must triage diverse threats. *Third*, extracting bidirectional flow statistics entirely in the data plane remains underexplored; most systems rely on unidirectional or packet-level features, or offload feature computation to an external server. *Fourth*, dimensionality reduction has been treated in isolation from the downstream classifier: although Planter [19] supports PCA as a standalone module, no prior work integrates PCA with a downstream classifier under a single range-match encoding, and no prior work makes the trade-off between a raw-feature classifier and a PCA-projected one explicit on hardware-relevant axes (per-entry match-key width, classifier-table size, and pipeline-stage placement). *Fifth*, translating a trained ML model into a deployable P4 program typically requires significant manual effort across separate tools for training, feature selection, table generation, and code synthesis.

In this paper, we address all five gaps through the following contributions:

- We deploy DT and RF classifiers as range-match P4 entries rather than exact-match entries, representing each tree leaf as a single rule with per-feature ranges. This shrinks the source-level (logical) classifier-table description by up to 51.71% relative to the exact-match strategy used by prior

in-network ML systems such as IIsy [18] and SwitchTree [9], and naturally expresses the axis-aligned splits of a decision tree (Section 2 discusses the on-chip resources the two encodings actually consume).

- We integrate PCA as an optional dimensionality-reduction stage that is itself encoded as range-match P4 tables and feeds the downstream classifier through the same encoding. To our knowledge, this is the first integration of PCA with a downstream classifier under a single range-match encoding in a P4 data plane. We characterize the trade-off introduced by this stage: PCA narrows the classifier match-key from 552 bits to $k \cdot b$ bits (≤ 320 bits across our grid) and gives the deployer a tunable footprint dial via k , at the cost of more classifier entries than a raw-feature range-match DT on the same dataset (Section 4.2).
- We perform multiclass classification directly in the data plane on the CIC-IoT 2023 dataset [12] (Benign, DoS, Brute Force, Reconnaissance), in contrast to the predominantly binary detectors in prior work [8,11,9].
- We extract 20 bidirectional flow features in the data plane across seven categories (protocol, ports, timing, volume, TCP flags, TCP window, cross-flow). This is substantially richer than the unidirectional or packet-level features used by prior in-network approaches [13,1,17].
- We provide an automated end-to-end pipeline from raw PCAP to deployable P4, covering feature extraction, model training, optional PCA projection, table-entry generation, and P4 code synthesis.

The full source code, training pipeline, and table-generation scripts used in this paper are publicly available at <https://github.com/vafekt/p4sec>.

2 Related Works

We analyse prior work along five dimensions: inference location, table encoding, feature extraction, classification granularity, and use of dimensionality reduction. Table 1 provides the structured comparison.

In-network ML inference: Xiong and Zilberman [16] first mapped DT, SVM, Naive Bayes, and K-means to match-action tables on BMv2 and NetFPGA SUME with 11 packet-level features. IIsy [18] added a hybrid Tofino architecture forwarding low-confidence packets to a backend; its discretized-feature encoding allocates one entry per discretized value, so table sizes scale with the number of distinct feature values. Planter [19] unified eleven ML algorithms under three mapping methodologies (direct, encode-based, and lookup) and includes PCA as a standalone module. As an ML-mapping framework it does not include a feature-extraction stage; it consumes pre-computed CSV features and does not integrate PCA with a downstream classifier under a single range-match encoding. SwitchTree [9] placed an RF of three trees in BMv2 with exact-match direct mapping (binary only on UNSW-NB15), Akem et al. [1] encoded RF on Tofino for encrypted-traffic classification with range/ternary entries, and Zang et al. [17] deployed DT/RF on IoT gateways through Planter with five unidirectional features. Ganesan and Sarac [5] use range encoding for an ensemble on

Table 1. Comparison of P4 based ML approaches. DP = Data Plane, Hy = Hybrid, B = Binary, M = Multiclass.

Work	Inference	Match Type	Bidir. Flow	Classes	DP Reduction
Xiong [16]	DP	Exact/Ternary	✗	M	✗
IIsy [18]	Hy	Exact/Ternary	✗	B	✗
Planter [19]	DP	Exact/Ternary	✗	M	PCA ^a
Euclid [8]	DP	Ternary	✗	B	✗
CD2P [15]	Hy	Not applied	✗	B	✗
HISTA [10]	Hy	Not applied	✗	B	✗
CO-STOP [4]	Hy	Not applied	✗	M	✗
Musumeci [11]	Hy	Not applied	✗	B	✗
SwitchTree [9]	DP	Exact	✓	B	✗
Akem [1]	DP	Ternary/Range	✗	M	✗
P4DDLe [3]	Hy	Exact	✓	B	✗
P4Pir [17]	DP	Exact	✗	M	✗
Ganesan [5]	DP	Range	✗	B	✗
Ours	DP	Range	✓	M	PCA^b

^aStandalone module, not integrated with classifier.^bIntegrated with downstream classifier via range match.

CIC-IDS2017 but only for binary classification, while attack-specific statistical approaches [8,15,10] rely on hand-tuned thresholds rather than learned boundaries. Hybrid systems [4,11,3] extract features in the data plane but run inference in the controller, reintroducing the latency that in-network processing aims to eliminate.

Encoding and dimensionality reduction: Exact-match encoding [9,19] requires one entry per discretized value and scales with the discretization grid; range/ternary encoding [1,5,19] collapses contiguous intervals into single rules by exploiting axis-aligned tree splits. Our work is the first to adopt range match for both the PCA projection and the downstream classifier, shrinking the source-level (logical) classifier-table description by up to 51.71% relative to exact match. Range and exact match consume different on-chip resources on programmable ASICs (range tables compile to TCAM after prefix expansion, while exact-match tables are placed in SRAM hash banks), so this reduction is on logical entry count, not a direct TCAM saving over exact match.

As Table 1 illustrates, no existing work simultaneously combines range-match PCA encoding, range-match tree classifiers, multiclass output with four or more classes, bidirectional flow features, and a fully automated PCAP-to-P4 pipeline. Planter [19] is the closest prior work, but the differences matter in practice: it treats PCA as a standalone module and uses an exact-match-based mapping for its tree classifier in the configuration most directly comparable to ours, so its table size scales with the number of distinct discretized feature values, while

our pipeline encodes *both* the PCA projection and the downstream tree classifier as range-match entries, representing each tree leaf as one rule with per-feature ranges and shrinking the source-level classifier rule entries.

3 Proposed Framework

The system operates in two phases. An *offline training phase* (Fig. 1) processes labelled captures through a five-step pipeline that trains the models and generates the P4 program with pre-populated table entries; an *online inference phase* (Fig. 2) then executes feature extraction, dimensionality reduction, and classification entirely in the data plane at line rate. Every numerical operation (canonical 5-tuple normalization, flow-feature accumulation, PCA code assignment, classification) is implemented identically in both phases, so the switch reproduces exactly the predictions observed during training.

3.1 Offline Training Phase

The offline pipeline (Fig. 1) transforms raw PCAP captures into a compiled P4 program and a set of table-entry commands. We use CIC-IoT 2023 [12], a recent benchmark generated on a physical testbed covering IoT attacks across seven categories, which provides raw PCAPs (required for header-level compilation). It comprises five steps, each implemented as a standalone Python script.

Step 1: Feature extraction

Raw PCAP files are parsed to extract bidirectional flow features into a CSV dataset. Each flow is identified by a canonical five-tuple in which the endpoint with the numerically smaller IP address is the forward source (equal IPs are disambiguated by port); the same normalization is replicated in the generated P4 code, so the offline extractor and the data plane assign the same direction label to every packet of a given flow. A CRC16 hash indexes the per-flow register arrays that hold the counters, timestamps, and maxima feeding Table 2; a CRC32 hash indexes a second Bloom-filter register for collision detection. On a slot collision, the incoming packet is accumulated on the existing flow (a small measurement-noise contribution acceptable because the per-flow counters are aggregates) and a `collision` bit is raised in the digest so the controller can log the event; the slot frees on idle timeout. Per packet, the switch first canonicalizes the 5-tuple and hashes it, then resets the per-flow registers if the flow is new or the idle timeout T_{idle} (default 20s) has elapsed, increments the directional packet/byte counters, packet-length maxima, and TCP-flag counters, and finally updates the inter-arrival-time and TCP-window extrema. A flow is finalized when a FIN or RST is observed (with at least one packet seen) or on idle timeout; only finalized flows proceed to classification.

Table 2 lists the 20 features in seven categories. They align with the flow-level statistics used by Zeek [14] and NetFlow/IPFIX-based detection [7]; separating forward and backward counters exposes the directional asymmetries that characterize flood, scan, and amplification attacks. A large fraction of the features

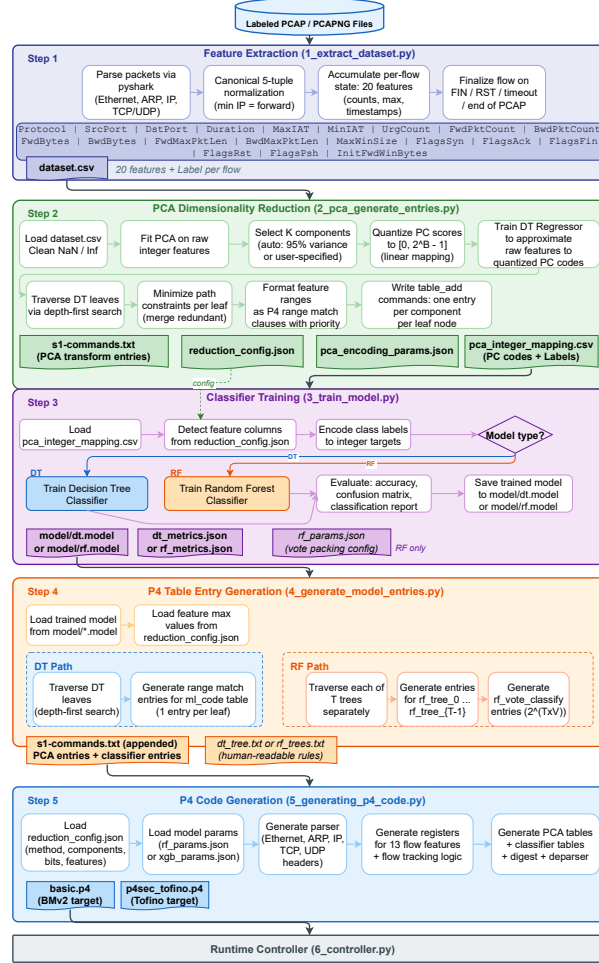


Fig. 1. Five-step offline training pipeline. A shared configuration file keeps feature handling consistent across all steps.

(flag counters, window-size extrema, initial advertised window, and the SYN-based cross-flow signal) target TCP-specific behaviour because the three attacks we target are predominantly TCP based in CIC-IoT 2023 and their signatures live in TCP control-plane patterns such as half-open bursts, repeated SYN/RST sequences, and anomalous initial windows; UDP and ICMP flows are still captured by the protocol-agnostic volume, timing, port, and flow-count features. Every feature is a count, sum, maximum, or minimum, requiring only addition and comparison, both of which are natively supported by P4.

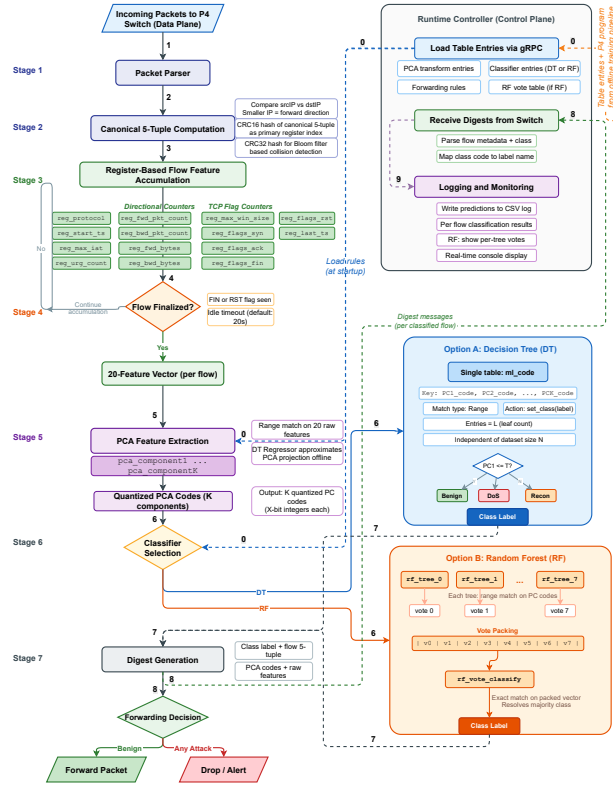


Fig. 2. Complete in-network inference pipeline in the data plane, Stages 1 - 7.

Step 2: PCA dimensionality reduction

The 20 raw features span heterogeneous bit widths (8 to 48 bits), producing a joint key of 552 bits at canonical P4 widths. Matching on this composite key directly is possible but inefficient: it requires either a very wide range-match table or splitting the classifier across multiple Tofino stages, and it ties the table layout to the specific feature set. A PCA-projected key avoids both pressures by keeping the classifier match-key at $k \cdot b$ bits, which on the configurations evaluated in Section 4.2 ranges from 80 bits ($k=5, b=16$) to 320 bits ($k=10, b=32$) and fits inside a single Tofino match unit on every (k, b) in our grid. PCA addresses this by projecting the feature vectors onto a K -dimensional subspace, with each component quantized to a B -bit integer:

$$code_j = \text{clamp}\left(\text{round}\left(\frac{pc_j - \min_j}{range_j} (2^B - 1)\right), 0, 2^B - 1\right). \quad (1)$$

A linearized PCA with quantized integer coefficients is in principle expressible on Tofino via its stateful-ALU multiplier, but the $20 \cdot K$ multiply-accumulates per

Table 2. Twenty bidirectional flow features extracted in the data plane, grouped by category.

Category	Feature	Description	Bits
Protocol	Protocol	IP protocol (TCP/UDP/ICMP)	8
Ports	SrcPort	Canonical source port	16
	DstPort	Canonical destination port	16
Timing	Duration	First-to-last packet time	48
	MaxIAT	Maximum inter-arrival time	48
Volume	FwdPktCount	Forward packet count	32
	BwdPktCount	Backward packet count	32
	FwdBytes	Forward byte count	32
	BwdBytes	Backward byte count	32
	FwdMaxPktLen	Max forward packet length	16
	BwdMaxPktLen	Max backward packet length	16
TCP Flags	FlagsSyn	SYN packet count	32
	FlagsAck	ACK packet count	32
	FlagsFin	FIN packet count	32
	FlagsRst	RST packet count	32
	FlagsPsh	PSH packet count	32
Window	MaxWinSize	Maximum TCP window size	16
	InitFwdWinBytes	Initial forward TCP window	16
Cross-flow	FlowCountPerSrc	Distinct flows seen per source	32
	SynCountPerDst	SYN packets seen per destination	32

packet (140 at $K=7$) compete for the small per-stage multiplier budget, must carry the 552-bit raw vector and partial sums in PHV across multiple ingress stages of projection, and are not natively expressible inside a v1model match-action unit on our primary BMv2 target. We therefore train a Decision Tree Regressor (DTR) offline to predict the K -dimensional quantized code from the 20 raw integer features, trading multiply-accumulate compute for table memory and keeping the projection inside a single range-match lookup that uses the same encoding as the downstream classifier. Each DTR leaf stores the mean quantized code of its training samples, partitioning the raw feature space into axis-aligned regions mapped to fixed PCA-code tuples. To emit P4 entries, the DTR is traversed depth-first: at each leaf the path constraints are minimized by keeping only the tightest bound per feature, and one range-match entry is written per component (clipped to the canonical P4 width $FMAX[f]$ for unconstrained features) with priority increasing with depth. A DTR with L leaves and K components therefore yields exactly $L \times K$ entries, independent of training-set size. The step writes four artifacts: table-entry commands, a universal pipeline

configuration file, quantization parameters, and a CSV of quantized codes with labels.

Step 3: Classifier training

The quantized PCA codes from Step 2 train either a DT or RF classifier (Scikit-learn). The feature columns and maximum values come from the shared configuration file, so the classifier operates on exactly the representation the switch will produce.

Step 4: Table entry generation

The trained classifier is converted to P4 range-match entries using the same DFS-with-tightest-bounds traversal as the PCA stage, but with class labels as outputs. The DT is encoded as a single range-match table whose keys are the K PCA codes; each leaf maps to one entry. A single range-match entry covers *every* feature combination within its leaf region, whereas an exact-match entry fires only when the input equals one of the stored training samples. For a tree with L leaves trained on N samples, range-match therefore produces exactly L entries with full input-space coverage within those leaves, while an exact-match deployment requires up to N entries (one per unique training feature vector) and still leaves gaps for any combination not seen at training time. On our 80/20 split with $N=136\,640$ unique training feature vectors, a raw-feature range-match DT trained without depth limits fits in 436 entries (Section 4.2); the deployed PCA+DT in our recommended window ($k \leq 7$) uses tens of thousands of entries, also below this exact-match upper bound, while larger k settings can exceed it as PCA leaves multiply.

The RF classifier extends this to T trees, each encoded as a separate range-match table with the same PCA-code keys. Each tree writes its prediction into $V = \lceil \log_2 C \rceil$ bits of a packed vote register, and a final exact-match table resolves the majority class from the concatenated $T \times V$ -bit vector. For our four-class setting ($C=4$, $V=2$) with $T=4$ trees, the vote table has $2^8=256$ entries, well within practical TCAM limits. The entire ensemble completes in a single pipeline pass without recirculation.

Step 5: P4 code generation

A code generator reads the pipeline configuration and model parameters and emits the complete P4 program: parser, register declarations, canonical hash actions (CRC16 for indexing, CRC32 for Bloom-filter collision detection), PCA transform tables, classifier tables, digest logic, and deparser. The current implementation targets BMv2 (v1model); a Tofino (TNA) extension is discussed in Section 5. The outputs are passed to the runtime controller, which compiles the P4 program and installs the rules.

3.2 In-Network Inference Phase

Once the runtime controller loads the compiled program and table entries into the switch via gRPC, packet processing proceeds entirely in the data plane. The

seven stages are shown in the left column of Fig. 2 and use only addition, min/-max, and conditional increments. A flow is finalized when a TCP FIN or RST flag is observed (with at least one packet) or after an idle timeout of 20 s. Finalized features feed the pre-installed range-match tables that emit the K quantized PCA codes and then the classifier: for DT, a single range-match table `ml_code` returns the label; for RF, each of the T trees votes through its own range-match table into a packed vote register that an exact-match table `rf_vote_classify` resolves to the majority class. The class label, 5-tuple, PCA codes, and raw feature vector are sent as a digest to the controller; the switch forwards benign traffic and drops or alerts on any attack class. The entire inference path completes in a single pipeline pass without recirculation.

4 Evaluation

4.1 Experimental Setup

We use a subset of CIC-IoT 2023 covering four classes (Benign, DoS, Brute Force, Reconnaissance), split into training (80%) and holdout (20%) subsets. We evaluate 24 configurations spanning three design parameters: the number of PCA components $k \in \{5, 6, 7, 8, 9, 10\}$, the quantization bit-width $b \in \{16, 32\}$, and the classifier type (DT or RF).

The testbed follows Fig. 2. A BMv2 switch (`simple_switch_grpc`) runs the compiled P4 program and connects two Mininet hosts that act as traffic source and sink. The source replays labelled PCAP traces with `tcpreplay`; the switch extracts the 20 bidirectional flow features, applies the quantized PCA transform, and runs the DT or RF classifier entirely in the data plane. Finalized flows are sent as gRPC digests to a Python control plane, which logs them for scoring against the ground-truth labels. We target BMv2 because it implements the complete v1model semantics without the stage, memory, and hash-width restrictions of current ASICs, which lets us isolate the classification quality and resource behaviour of the proposed design from target-specific trade-offs; hardware deployment on Intel Tofino is discussed in Section 5. For each configuration we report macro-averaged F1 and per-class F1 on BMv2, along with three resource metrics (total table entries, rules-file size in MB, and rule-loading time in seconds).

4.2 Results

We organize the results around four questions. First, how does detection quality on the BMv2 data plane vary with the number of PCA components k , the per-component quantization width b , and the classifier (DT or RF)? Second, how does that quality break down per attack class? Third, what is the resource cost (range-match entries, rules-file size, rule-loading time) of each configuration, and how does it scale with k ? Fourth, does the PCA stage itself pay off relative to a no-reduction baseline on hardware-relevant metrics that are not visible from

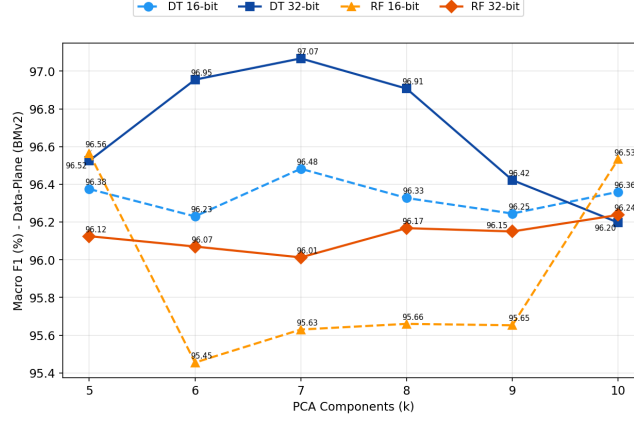


Fig. 3. Macro F1 on the BMv2 data plane as a function of PCA components k , for the four $(b, \text{classifier})$ groups.

macro-F1? Each question is answered in turn below using the metrics introduced in Section 4.1.

Macro F1 across configurations

Fig. 3 shows the macro-averaged F1 on BMv2 for the four $(b, \text{classifier})$ groups as k grows from 5 to 10. Three structural observations emerge. First, on average DT is above RF (mean 96.51% vs 96.02%, a 0.49 pp gap over the 12 (k, b) configurations) and 32-bit is above 16-bit, but the ordering flips at the curve extremes (RF overtakes DT at $k=5, b=16$ and at $k=10$ for both bit-widths; 16-bit overtakes 32-bit for DT at $k=10$ and for RF at $k=5$ and $k=10$), so neither is a strict per- (k, b) Pareto improvement. The DT-over-RF gap reflects that on quantized PCA codes a single deeper tree captures the available signal better than a four-tree majority vote, which after quantization tends to cast highly correlated votes and therefore offers little ensemble benefit while still paying the entry-count overhead documented below. Second, widening the per-component quantization from 16 to 32 bits lifts macro F1 by 0.28 pp on average; the lift is largest at intermediate k and turns slightly negative at the boundaries, where the wider codes also enlarge the rule set and can place a leaf on a noisier point of the validation distribution. Third, the DT curves rise steeply between $k=5$ and $k=7$ and then plateau: the best operating point is $k=7, b=32$, DT at 97.07% macro F1, beyond which extra components add resource cost without measurable F1 gain. The plateau is consistent with the per-component variance of the trained PCA, which declines rapidly past the seventh component and contributes progressively less signal as k grows. A practitioner can therefore stop at $k=5, b=32$, DT (96.52%, only 0.55 pp below the peak) without crossing a quality cliff.

Per-class analysis

The macro-F1 figure flattens four very different per-class behaviours into one number, so Fig. 4 reports per-class F1 on BMv2 across all (k, b) settings, with

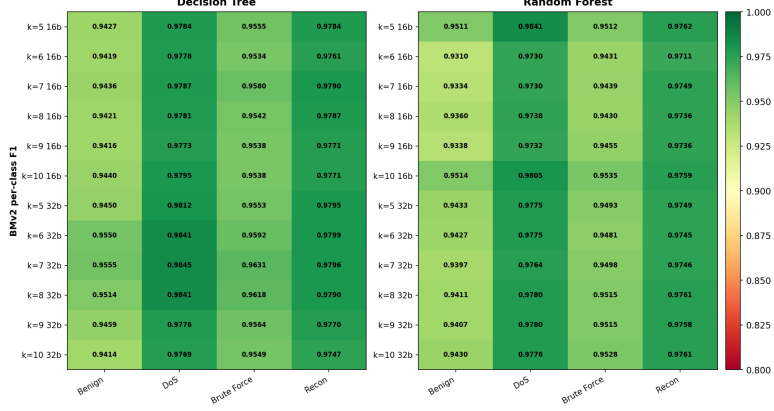


Fig. 4. Per-class F1 on BMv2 across all (k, b) settings; left DT, right RF.

DT on the left and RF on the right and a colour scale. The four classes split into a clear difficulty hierarchy. DoS and Reconnaissance are the easiest: DT reaches F1 in 0.9769–0.9845 on DoS and 0.9747–0.9799 on Reconnaissance. The reason is that the corresponding flows have distinctive volumetric and scanning fingerprints (sustained high packet rate for DoS, short low-byte probes spread across many destinations for Reconnaissance) that survive PCA projection because they project onto the dominant components. Brute Force sits in the middle (DT F1 0.9534–0.9631): it shares timing characteristics with both Benign sessions (legitimate repeated logins) and with low-rate Reconnaissance, so a fraction of its flows ends up near the decision boundary. Benign is consistently the hardest class: DT Benign F1 lies in 0.9414–0.9555, and RF drops to 0.9310 in the worst case. Two factors compound here. Benign flows are the most heterogeneous class (interactive sessions, bulk transfers, and short control exchanges all share the label), so they occupy a larger volume of feature space than any single attack class; and quantization rounds many of those Benign points onto the same codes already used by neighbouring attack classes, where range tables cannot resolve them further. Lower Benign F1 is also the operationally costly failure mode because each misclassified Benign flow on the attack side means an attack escaping as legitimate.

The 16→32-bit transition acts non-uniformly across classes. At most (k, b) settings the wider quantization lifts DT Benign F1 by up to +1.31 pp and DoS F1 by up to +0.63 pp, since the extra bits sharpen leaf boundaries in the densest regions of the feature space. On a few configurations the 32-bit run is slightly below the 16-bit one, however, because the wider codes also enlarge the rule set and can place a boundary on a noisier point of the validation distribution. The take-away is that 32-bit quantization helps on the hardest classes on average but is not a strict Pareto improvement at every (k, b) , which is consistent with the small 0.28 pp average lift in macro F1.

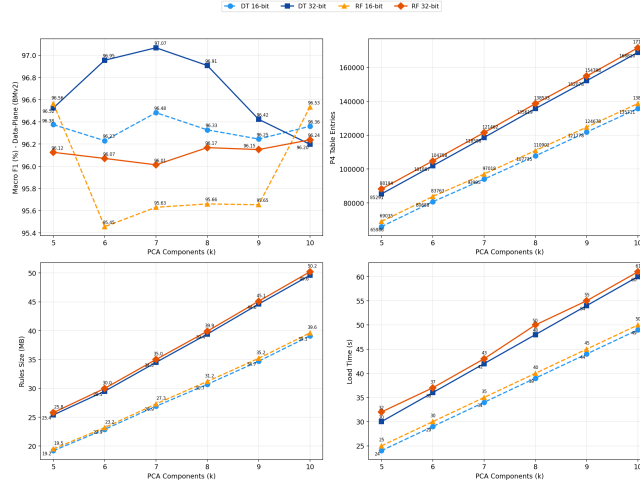


Fig. 5. BMv2 macro F1, P4 table entries, rules-file size, and loading time as functions of k .

Resource scaling

Fig. 5 reports the resource cost of each (k, b) configuration alongside macro F1. The dominant trend is that table entries grow linearly in k : each additional component adds 13.3–14.7k entries at 16-bit quantization and 16.5–17.0k at 32-bit. The slope is steeper at 32 bits because each new component splits more bins along its axis, so the DT learns finer leaves whose ranges expand into more prefix entries. DT 16-bit at $k=5$ uses 65 986 entries (19.2 MB of rules, 24s to load), and $k=10$ uses 135 711 entries (39.1 MB, 49s), a $2.06\times$ growth in entries that is tracked one-to-one by both rule-file size and rule-loading time, indicating that loading is dominated by per-rule installation rather than per-byte parsing. RF adds 2.7–3.2k extra entries over DT at the same (k, b) because each of its trees contributes its own leaves to the merged range table, yet, as shown in the macro-F1 discussion, RF does not beat DT on average across the 12 (k, b) configurations, so the RF surcharge does not pay off on average on this dataset.

Reading Figs. 3 and 5 together exposes a clean three-way Pareto frontier inside the DT family. The high-quality point $k=7, b=32$, DT delivers the peak 97.07% macro F1 at 118 588 entries; the balanced point $k=5, b=32$, DT cuts entries by 28% to 85 291 for a 0.55 pp F1 loss; and the compact point $k=5, b=16$, DT lands at 65 986 entries and 96.38% F1, the smallest deployable footprint at -0.69 pp from the peak. Compared with the $N=136\,640$ -entry exact-match baseline that prior in-network ML systems would require, this compact PCA+DT point delivers a 51.71% source-level entry reduction $(1 - 65\,986/136\,640)$. Because all three points share the DT family, a single deployment script can move along this frontier by retraining with a different (k, b) rather than swapping classifiers, which is convenient when target hardware imposes a hard table-size cap.

Table 3. Deployed PCA+DT ($k=7, b=32$) vs raw-feature range-match DT on the same dataset and hyperparameters. The last column names the better pipeline and reports the relative reduction it achieves; *tied* means the difference is below measurement noise.

Metric	Raw DT	PCA+DT	Better pipeline (rel. reduction)
P4 entries	436	118 588	Raw (99.6% fewer entries)
Key width (bits)	552	224	PCA (59% narrower key)
Rules file	0.12 MB	34.5 MB	Raw (99.7% smaller file)
Load time	< 1 s	42 s	Raw (faster load)
Macro-F1	0.9709	0.9707	tied (within 0.02 pp)

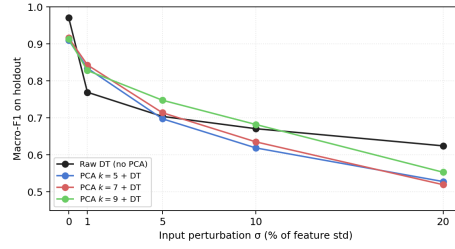


Fig. 6. Macro-F1 under Gaussian input noise of σ % of feature std (mean over 5 seeds).

Range-match DT vs PCA+DT: a measured trade-off

To put the PCA pipeline in context, we compare it against a *raw-feature* DT baseline that uses the same range-match encoding but skips PCA entirely. The baseline is trained on the same 20-feature working set used by the deployed pipeline (Table 2) under the same 80/20 stratified split and the same DT hyperparameters as the deployed PCA+DT pipeline. Table 3 reports the side-by-side numbers; the PCA+DT row matches the deployed $k=7, b=32$ row of Fig. 5.

Where raw-feature range-match wins. On this dataset the 20 raw features are individually highly discriminative, and a DT reaches near-pure leaves with only 436 splits. Range-match expansion therefore produces a much smaller classifier table and an essentially tied macro-F1 against PCA+DT, which makes the raw-feature pipeline a natural choice when the binding constraint is total entries or rule-loading latency.

Where PCA pays off. PCA still buys three properties that the raw-feature pipeline does not have, and that matter when the binding constraint is hardware match-key budget or input-quality robustness rather than entry count.

First, using PCA gets a narrower per-entry match-key. Summing canonical P4 widths over the 20 raw features (e.g., `Duration` 48 b, `FwdPktCount` 32 b, `Protocol` 8 b) gives $W_{\text{raw}}=552$ b, while the PCA classifier keys only on $W_{\text{pca}}=k \cdot b$ b (160 b at $k=5, b=32$; 224 b at $k=7$; 288 b at $k=9$): a 48–71% width reduction depending on k . This narrowing clears Tofino’s per-stage match-key budget within a single match unit, whereas the 552-bit raw composite would have to be split or compressed across pipeline stages to enter TCAM at all. Clearing

the match-width gate is a necessary precondition for hardware deployment but not sufficient by itself. Stage allocation, table placement, PHV usage, and action complexity must also fit, as discussed in Section 5.

Second, the raw-feature pipeline has a single operating point (436 entries, $W=552$ b, 0.9709 F1). Choosing $k \in \{5, \dots, 10\}$ and $b \in \{16, 32\}$ gives the PCA pipeline a continuous ladder of 24 operating points (Fig. 5); the deployer can match any target’s per-stage match-key budget W^* by picking (k, b) such that $k \cdot b \leq W^*$ before retraining. The raw pipeline has no such dial: its key width is set by the canonical P4 widths of the chosen features.

Third, there is graceful degradation under light input perturbation. The features observed by the data plane are not exact copies of the offline training values: CRC-16 hash collisions in the per-flow register array can cross-contaminate counters between two coexisting flows; sub-second clock granularity adds jitter to the timing features (*Duration*, *MaxIAT*); and idle-timeout-induced reporting can truncate the counters of a still-active flow. We model these collectively, as a smoothness sanity check rather than as an adversarial threat, by perturbing each test input with Gaussian noise of $\sigma \in \{1, 5, 10, 20\}\%$ of the per-feature standard deviation (five seeds each). Fig. 6 reports macro-F1 of an sklearn DT on the 20 raw features against sklearn PCA+DT pipelines for $k \in \{5, 7, 9\}$. Although the raw DT scores slightly higher on clean inputs (0.971 vs 0.91–0.92), it drops sharply at the smallest perturbation: it retains 79% of its clean F1 at $\sigma=1\%$, while PCA+DT $k=7$ retains 92%, a ~ 13 pp relative-retention advantage. The advantage shrinks to ~ 5 pp at $\sigma=5\%$, the curves are tied at $\sigma=10\%$, and at $\sigma=20\%$ the raw DT is more robust. PCA therefore helps under *light* measurement noise through averaging (each component is a linear combination of multiple raw features, so per-feature perturbations partially cancel) but not under heavy or adversarial perturbation.

The summary is therefore: The proposed encoding is what makes the design fit programmable hardware; the raw-feature DT is the smaller and essentially equally accurate option whenever the target can absorb a 552-bit composite key in one stage; and PCA is the option when the per-stage match-key budget is binding, when a tunable footprint dial through (k, b) is useful, or when light input noise is a concern, paying for these with more entries and a larger rules file. The recommended PCA operating points ($k \in \{5, 6, 7\}$, $b=32$, DT) fit comfortably within BMv2 resources and leave headroom for the additional Tofino constraints discussed in Section 5.

5 Towards Hardware Deployment on Intel Tofino

BMv2 implements full v1model semantics, but real deployment demands mapping the design onto an ASIC such as Intel Tofino, which imposes strict constraints on pipeline depth, on-chip memory, match resources, and action complexity. The hardware resource pressure of our pipeline (Fig. 2) comes from three sources. First, stateful flow storage in Stages 2–3: each active flow occupies register slots holding the inference features, with CRC-16 hash indexing, so SRAM

Table 4. Feature bit allocation per PCA table.

Field	Bits	Field	Bits
<i>protocol</i>	8	<i>flags_1</i> (SYN/ACK)	16
<i>src_port+dst_port</i>	32	<i>flags_2</i> (PSH)	16
<i>duration+max_iat</i>	96	<i>flags_3</i> (FIN/RST)	8
<i>fwd_pkt+bwd_pkt</i>	32	<i>fwd_len+bwd_len</i>	32
<i>fwd_bytes+bwd_bytes</i>	32	<i>init_fwd_win</i>	16
<i>max_win_size</i>	16	Total	304

scales as $M_{\text{total}} = N_{\text{hash}}^{\text{active}} \times M_{\text{per-flow}}$ and a compact feature layout is essential. Second, match-action resources in Stages 5–6: the PCA projection and the classifier are both range tables that compile to TCAM, so the design is bounded by entry count, match-key width, and number of features per table; beyond raw TCAM volume, the compiler must also place these tables within the 12 ingress stages of Tofino 1 without violating ALU, crossbar, or data-dependency budgets. Third, control-plane reporting in Stage 7: Tofino digests are designed for lightweight notifications, so we trigger them event-driven (flow termination, idle timeout, or suspicious classification) rather than per packet.

To probe deployability, we map the K PCA components onto independent range tables sharing one feature vector. Table 4 gives the per-table key layout: counters and flag indicators use the narrowest widths that still cover the per-flow ranges observed in CIC-IoT 2023, the five TCP-flag counters are packed into three composite fields, and the two cross-flow features from Table 2 (*FlowCountPerSrc* and *SynCountPerDst*) are excluded because they require maintaining shared state across distinct per-source and per-destination index spaces, which complicates single-stage placement on Tofino 1. The packed key totals 304 bits.

This width sits below Tofino 1’s per-stage match-key budget, so the PCA module clears the match-width gate. Match width alone, however, does not guarantee end-to-end compilation: stage allocation, table placement, PHV usage, action complexity, and interactions with surrounding pipeline components must also fit. The current implementation passes the match-width and SRAM checks but does not yet compile end-to-end on Tofino 1; ongoing work targets tighter feature selection (retaining fewer, more informative inputs) to relieve match-action pressure and complete the hardware port.

6 Conclusion and Future Work

This paper presented an automated pipeline for tree-based multiclass IoT attack detection entirely inside a P4 data plane. The central design decision is to encode the classifier on the 20 bidirectional flow features (Table 2) as a range-match table, representing each tree leaf as a single rule with per-feature ranges and shrinking the source-level (logical) classifier-table description by up to 51.71% relative to the per-sample exact-match encoding used by prior work (Table 1). DT beats RF on average in quantized form (mean 96.51% vs 96.02%), and table entries

grow linearly with k . PCA is offered as an optional preprocessing stage that is itself range-match-encoded and feeds the same classifier: across 24 PCA configurations on CIC-IoT 2023 it sustains macro F1 between 95.45% and 97.07%, with the best F1 point ($k=7$, $b=32$, DT) at 97.07% on 118 588 entries and the compact point ($k=5$, $b=16$, DT) at 96.38% on 65 986 entries, the configuration that delivers the 51.71% reduction over the 136 640-entry exact-match baseline noted above; a raw-feature range-match DT on the same 20-feature working set reaches 97.09% on 436 entries with a 552-bit composite key. PCA trades a larger classifier table for a 59% narrower per-entry match-key (224 vs 552 bits at $k=7$, $b=32$) and better robustness to light input perturbation. A Tofino resource analysis shows that the packed PCA layout passes the match-width and SRAM gates of Tofino 1, with end-to-end compilation pending tighter feature selection. The pipeline therefore offers two operating modes: a raw-feature range-match DT for footprint-bound targets, and a PCA-projected variant for targets whose binding constraint is the per-stage match-key width. Future work will realize the pipeline on the UfiSpace S9180-32X / BFN-T10-032D Tofino testbed (Section 5) for per-stage TCAM/SRAM occupancy and 100 Gbps validation, extend evaluation to the remaining CIC-IoT 2023 classes and to UNSW-NB15, CIC-IDS 2017, and EDGE-IIoTset, and add controller-driven incremental rule updates so the switch can absorb model refreshes without stopping the data plane.

Acknowledgments. This work was supported by the Ministry of Industry and Trade of the Czech Republic under Grant No. CZ.01.01.01/01/24_062/0007470. It was also funded in part by the German Research Foundation (DFG, Deutsche Forschungsgemeinschaft) under Germany’s Excellence Strategy-EXC 2050/2, Project ID 390696704 (Cluster of Excellence “Centre for Tactile Internet with Human-in-the-Loop” (CeTI), TU Dresden), and by the Federal Ministry of Education and Research of Germany in the programme of "Souverän. Digital. Vernetzt.". Joint project 6G-life, project identification number: 16KIS2413K.

References

1. Akem, A.T.J., Fraysse, G., Fiore, M.: Encrypted traffic classification at line rate in programmable switches with machine learning (2024). <https://doi.org/10.1109/NOMS59830.2024.10575394>
2. Bosshart, P., Gibb, G., Kim, H.S., Varghese, G., McKeown, N., Izzard, M., Mujica, F., Horowitz, M.: Forwarding metamorphosis: fast programmable match-action processing in hardware for sdn **43**(4), 99–110 (Aug 2013). <https://doi.org/10.1145/2534169.2486011>
3. Doriguzzi-Corin, R., Knob, L.A.D., Mendozzi, L., Siracusa, D., Savi, M.: Introducing packet-level analysis in programmable data planes to advance network intrusion detection. *Computer Networks* **239** (2024). <https://doi.org/10.1016/j.comnet.2023.110162>
4. El-Sayed, A., Toony, A.A., Alqahtani, F., Alginahi, Y., Said, W.: Co-stop: A robust p4-powered adaptive framework for comprehensive detection and mitigation of coordinated and multi-faceted attacks in sd-iot networks. *Computers and Security* **151** (2025). <https://doi.org/10.1016/j.cose.2025.104349>

5. Ganesan, A., Sarac, K.: Adversarial evasion attack detection using ensemble classifiers in p4 data plane. *Journal of Network and Systems Management* **33**(4) (2025). <https://doi.org/10.1007/s10922-025-09948-7>
6. Gao, Y., Wang, Z.: A review of p4 programmable data planes for network security. *Mobile Information Systems* **2021** (2021). <https://doi.org/10.1155/2021/1257046>
7. Hofstede, R., Čeleda, P., Trammell, B., Drago, I., Sadre, R., Sperotto, A., Pras, A.: Flow monitoring explained: From packet capture to data analysis with netflow and ipfix. *IEEE Communications Surveys and Tutorials* **16**(4), 2037 – 2064 (2014). <https://doi.org/10.1109/COMST.2014.2321898>
8. Ilha, A.D.S., Lapolli, A.C., Marques, J.A., Gaspar, L.P.: Euclid: A fully in-network, p4-based approach for real-time ddos attack detection and mitigation. *IEEE Transactions on Network and Service Management* **18**(3), 3121 – 3139 (2021). <https://doi.org/10.1109/TNSM.2020.3048265>
9. Lee, J.H., Singh, K.: Switchtree: in-network computing and traffic analyses with random forests. *Neural Computing and Applications* **37**(28), 23143 – 23154 (2025). <https://doi.org/10.1007/s00521-020-05440-2>
10. Li, D.C., Tu, H.H., Chou, L.D.: Cross-layer detection and defence mechanism against ddos and drdos attacks in software-defined networks using p4 switches. *Computers and Electrical Engineering* **118** (2024). <https://doi.org/10.1016/j.compeleceng.2024.109307>
11. Musumeci, F., Fidanci, A.C., Paolucci, F., Cugini, F., Tornatore, M.: Machine-learning-enabled ddos attacks detection in p4 programmable networks. *Journal of Network and Systems Management* **30**(1) (2022). <https://doi.org/10.1007/s10922-021-09633-5>
12. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors* **23**(13) (2023). <https://doi.org/10.3390/s23135941>
13. Nguyen, H.N., Nguyen, M.D., Montes De Oca, E.: A framework for in-network inference using p4 (2024). <https://doi.org/10.1145/3664476.3670453>
14. Paxson, V.: Bro: A system for detecting network intruders in real-time (1998)
15. Wang, Y.C., Su, P.Y.: Collaborative defense against hybrid network attacks by sdn controllers and p4 switches. *IEEE Transactions on Network Science and Engineering* **11**(2), 1480 – 1495 (2024). <https://doi.org/10.1109/TNSE.2023.3324329>
16. Xiong, Z., Zilberman, N.: Do switches dream of machine learning?: Toward in-network classification. p. 25 – 33 (2019). <https://doi.org/10.1145/3365609.3365864>
17. Zang, M., Zheng, C., Dittmann, L., Zilberman, N.: Toward continuous threat defense: In-network traffic analysis for iot gateways. *IEEE Internet of Things Journal* **11**(6), 9244 – 9257 (2024). <https://doi.org/10.1109/JIOT.2023.3323771>
18. Zheng, C., Xiong, Z., Bui, T.T., Kaupmees, S., Bensoussane, R., Bernabeu, A., Vargaftik, S., Ben-Itzhak, Y., Zilberman, N.: Iisy: Hybrid in-network classification using programmable switches. *IEEE/ACM Transactions on Networking* **32**(3), 2555 – 2570 (2024). <https://doi.org/10.1109/TNET.2024.3364757>
19. Zheng, C., Zang, M., Hong, X., Perreault, L., Bensoussane, R., Vargaftik, S., Ben-Itzhak, Y., Zilberman, N.: Planter: Rapid prototyping of in-network machine learning inference. *Computer Communication Review* **54**(1), 2 – 20 (2024). <https://doi.org/10.1145/3687230.3687232>